

OODM, Assignment 1

1) What is the role of notations in a modeling tool used for software design? Describe any 5 commonly used UML notations explaining each in brief.

= Notations are standardized graphical symbols used in software modeling languages such as UML. They help represent the structure, behavior and interactions of a software system in a clear and consistent manner.

Using standard notations improves communication among developers, designers, analysts and stakeholders, reduces ambiguity and serves as documentation throughout the SDLC.

Five commonly used UML notations are:

a) Class notation:

→ ~~A class~~ It represents a class in the system.

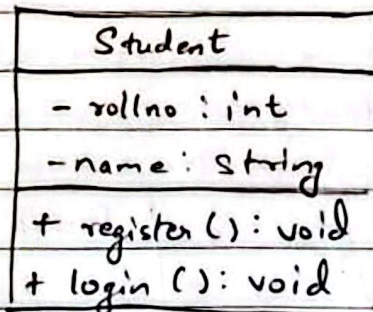
→ It is shown by a rectangle which is divided into three parts:

→ class name

→ attributes

→ Methods

Example:



b) Actor notation:

→ It represents an external user or another system that interacts with the system. It is ^{mainly} used in use case diagrams, sequence diagrams, system sequence diagrams.

→ It is drawn as a stick figure.

→ Eg: 
customer

iii) Use case notation:

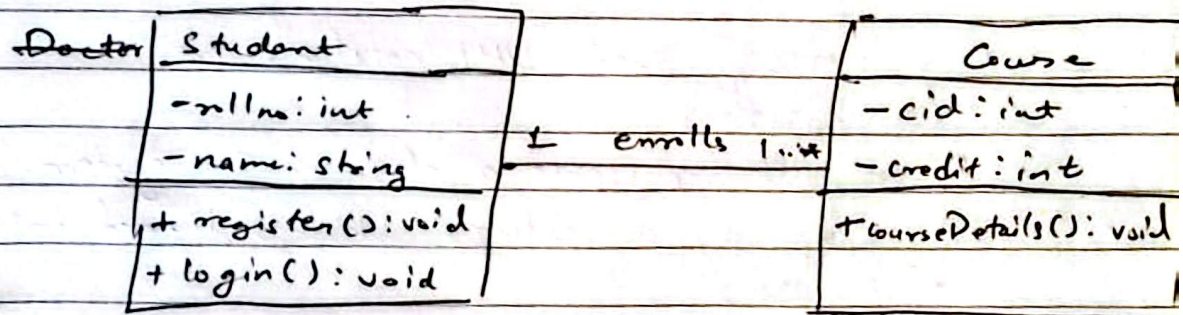
- It represents a function or service provided by the system.
- It is drawn as an oval. It ~~connects~~ is connected to actors to show interactions.

Eg: place order

iv) Association notation:

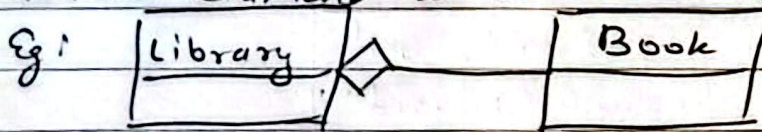
- It represents a relationship between two classes or between ~~an object/class with another object/class~~, an actor and a use case.
- It is simply drawn as a solid line.
- It may include multiplicity (eg: 1, *, 0..1, 0..*)

Eg:



v) Aggregation notation:

- Represents a whole-part relationship where the part can exist independently of the whole. It is shown by hollow diamond at the whole end.



this means library has books, but the book can exist even if library is removed.

2) Mention the significance of analysis and design in software development lifecycle from an object oriented perspective.

= OOA focuses on understanding and modeling the system requirements using objects, classes and their interactions. OOD transforms the analysis model into a detailed design that can be implemented in code. Together they play a vital role in developing high quality software.

Significance:

i) Better understanding of requirements:

- OOA identifies the system's objects, classes, attributes and relationships based on real world entities.
- This ensures that the user requirements are accurately captured before development begins.

ii) Improved system design:

- OOD provides a blueprint for implementation by defining class structures, methods, interfaces and object interactions.
- It results in a well organized and modular software architecture.

iii) Promotes reusability:

- Object oriented concepts such as inheritance, polymorphism, and encapsulation encourage code reuse. Existing classes and components can be reused in other applications, reducing development time and cost.

iv) Enhances modularity and easier maintenance:

- Each object represents a specific responsibility, making the system easier to understand, test and manage. Modules can be developed and tested independently.
- Since software is divided into independent classes and objects, changes in one module have minimal impact on others. This makes debugging, updating and extending the system easier.

v) It reduces development risks:

- Errors and inconsistencies can be identified during the analysis and design phases before coding begins. Early detection reduces the cost and effort of fixing defects.

vi) Supports scalability and flexibility:

- OOD's are easier to extend by adding new classes or modifying existing ones without affecting the entire system. This allows the software to adapt to future requirements.

vii) Improves communication:

- UML diagrams such as use case, class, sequence and activity diagram provide a common visual language. Developers, analysts, designers and stakeholders can communicate more effectively.

3) Explain software development models:

i) Waterfall model:

- The waterfall model is a linear and sequential software development model in which each phase must be completed before moving to the next phase.

Phases:

a) Requirement analysis:

- Gather and analyze all customer requirements.
- Prepare the Software Requirement Specification (SRS)

b) System design:

- Design the system architecture, database, user interface and software components based on the requirements.

c) Implementation:

- Developers write the program according to the design specifications.
- Individual modules are developed.

d) Testing:

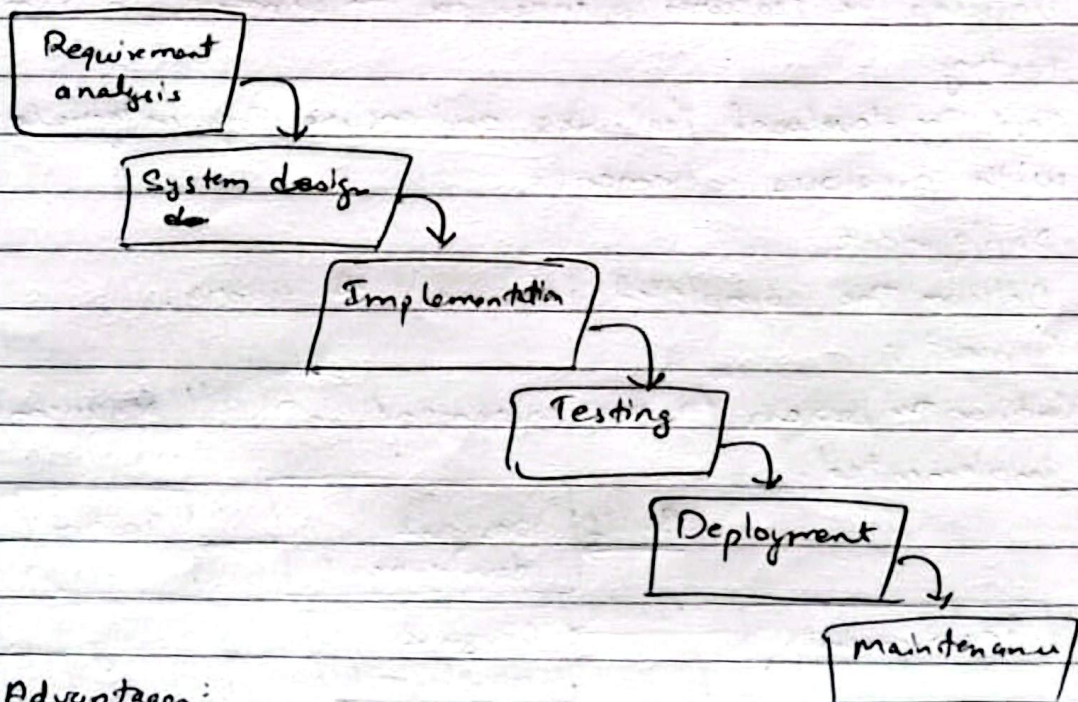
- The software is tested to identify and fix defects.
- It includes unit testing, integration testing, system testing and acceptance testing.

e) Deployment:

- The completed software is installed and delivered to the customer for use.

f) Maintenance:

- Corrects bugs, improves performance and updates the software after deployment.



Advantages:

- Simple & easy to understand.
- Easy to manage because each phase has clear deliverables.
- Well suited for small projects with stable and well defined requirements.

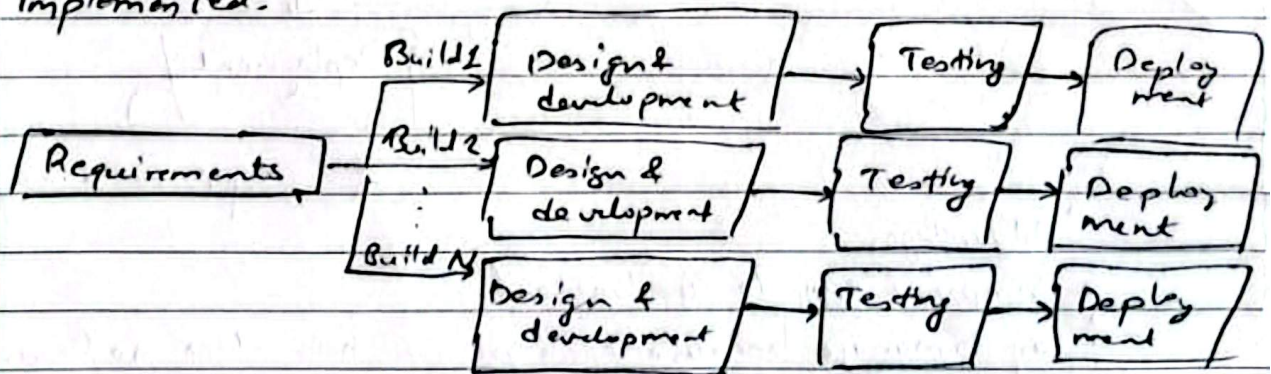
Disadvantages:

- Difficult to accommodate changes once development has started.
- Working software is available only at the end of the project.
- Not suitable for large, complex or frequently changing projects.

ii) Incremental model:

→ It is a software development model in which the system is developed and delivered in small, manageable parts called increments. Each increment adds new functionality to the previous version until the complete system is built.
Phases:

- Requirement analysis:**
→ Identify and prioritize requirements for the current increment.
- Design**
→ Design the architecture and components for the selected features.
- Implementation:**
→ Develop the features planned for the current increment.
- Testing:**
→ Test the developed features and ensure they integrate correctly with previous increments.
- Deployment:**
→ Deliver the completed increment to users.
- Repeat:**
→ Continue the process for next increment until all requirements are implemented.



Advantages:

- Working software is delivered early.
- Easy to accommodate changes in later increments.
- Easier to test and debug modules.
- Customer feedback can be incorporated after each release.

Disadvantages:

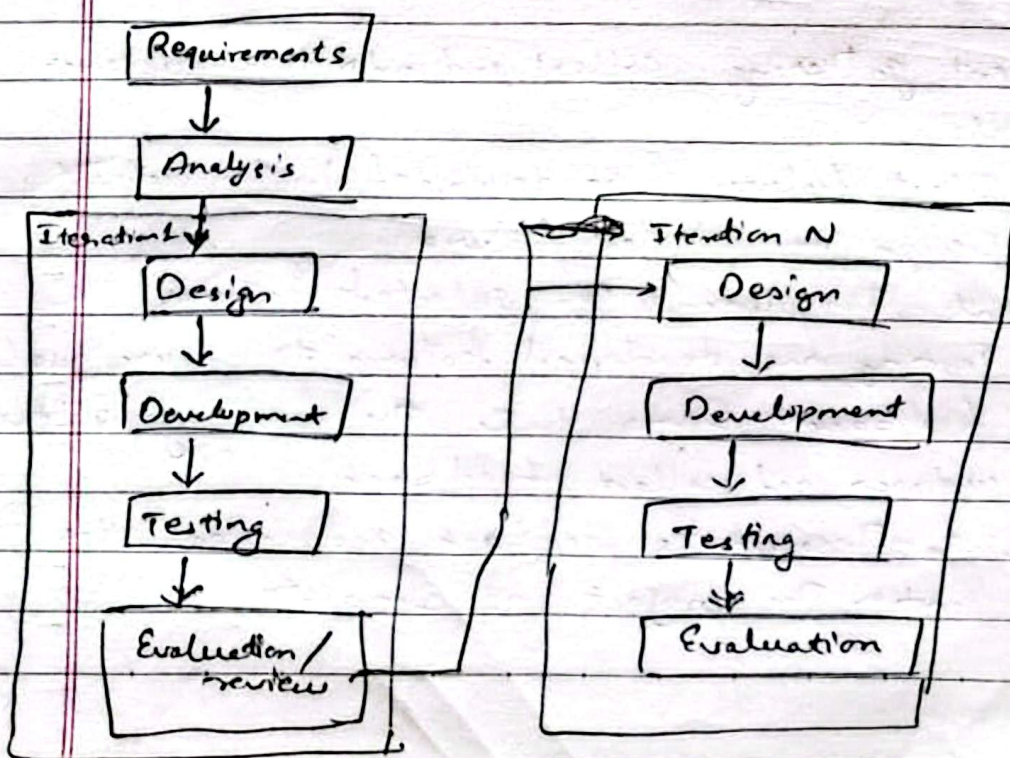
- Requires careful planning and system architecture.
- Overall project cost may be higher.
- Not suitable if system cannot be divided into independent increments.

iii) Iterative model:

- It is a software development model in which the software is developed through repeated cycle (iterations). Each iteration produces a working version of the software, which is refined and improved based on user feedback until the final product is completed.

Phases:

- Requirement analysis: Identify and analyze the requirements for the current iteration.
- Design: Create or improve the system design.
- Implementation: Develop the required functionality.
- Testing: Test the implemented features and fix defects.
- Evaluation: Gather user feedback and evaluate the feature.
- Repeat.



Advantages:

- Working software is available early.
- Easy to incorporate changing requirements.
- Customer feedback improves the final product.
- Easier to testing and debugging due to smaller development cycles.

Disadvantages:

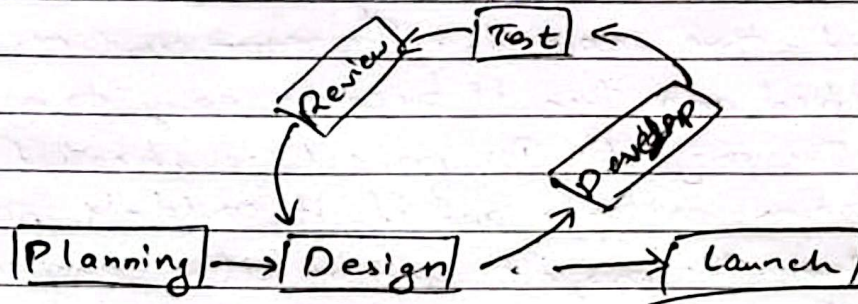
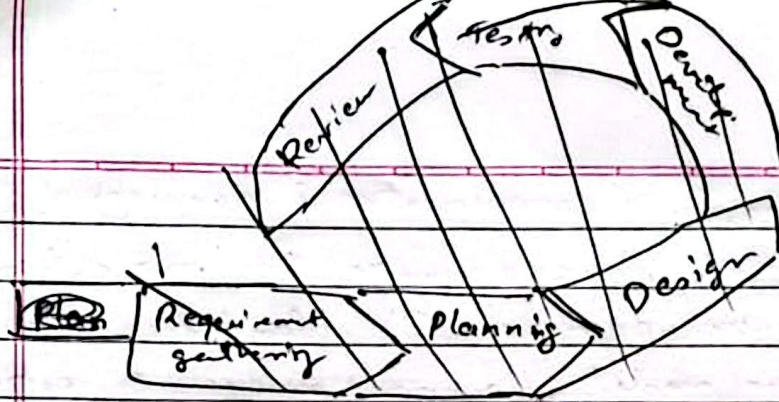
- Requires active customer involvement.
- Project management is more complex than waterfall.
- Total development cost and time may increase due to repeated iterations.

4.) Explain agile software development model with advantages & disadvantages.

= The agile software development model is an iterative and incremental approach to software development in which the project is divided into small iterations (called sprints). Each sprint produces a working version of the software, allowing continuous customer feedback and frequent ~~feedbacks~~ improvements throughout the development process.

Phases:

- a) Requirement gathering: Collect and prioritize customer requirements.
- b) Planning: Select features to be developed in the current sprint.
- c) Design: Design the selected features.
- d) Development: Implement the selected features.
- e) Testing: Testing the developed features to ensure quality.
- f) Review & feedback: Demonstrate the working software to the customer and collect feedback.
- g) Next sprint: Incorporate feedback and repeat the process until the project is complete.



Advantages:

- Delivers working software quickly and frequently.
- Easily accommodates changing customer requirements.
- Early testing helps identify and fix defects quickly.
- Reduces project risk through incremental development.

Disadvantages:

- Requires active customer participation throughout the project.
- Less emphasis on detailed documentation.
- Frequent requirement changes can increase development effort.
- Not suitable for projects with fixed requirements or strict regulatory documentation.

5) Explain phases of unified process in brief. Also highlight how each contributes to software development.

→ The Unified process is an iterative and incremental software development ~~into~~ ^{four} process. It divides software development into four phases, where each phase has a specific objective and contributes to the successful development of the software.

Phases:

i) Inception phase: (Communication & planning)

→ This is the initial phase where the project's scope and objective are defined. It identifies the scope using a use case model allowing managers to estimate costs and time required. The major customer requirements are identified and then it becomes easy to make a plan for the project. The project is checked against the milestone criteria and if it couldn't pass the criteria then the project can be either canceled or redesigned.

→ Contribution to software development:

- Defines the project vision and scope.
- Identify key stakeholders and major use cases.
- Evaluates project feasibility and risks.

ii) Elaboration phase: (planning & modeling)

→ A detailed evaluation and development plan is carried out and diminishes the risk. Revise or redefine the use case model, business case and risks. The software architecture is designed and major risks are addressed. Core functionalities are developed as prototypes if necessary.

→ Contribution to software development:

- Produces a stable software architecture.
- Clarifies detailed functional and non functional requirements.
- Reduces technical and project risks.
- Provides a solid foundation for implementation.

iii) Construction phase: (Construction) & ~~the~~

→ The actual coding, testing and integration of the software takes place. The system is developed incrementally until it is feature-complete.

Contribution to software development:

- Converts the design into a working software project
- Implements all planned features.
- Performs unit, integration and system testing.
- Produces a deployable software design.

Transition phase: (Construction & deployment)

The completed software is delivered to users. Installation, user training, acceptance testing, bug fixing, maintenance and final deployment are carried out. The development team begins to shrink and control is moved to maintenance team.

Contribution to software development:

- Delivers the software to the customer.
- Ensures the system meets user expectations.
- Resolves any remaining defects.
- Provides user support and training for successful adoption.

Explain difference between OOA and OOD with example.

Object oriented Analysis	Object oriented Design
Focuses on what the system should do	→ Focuses on how the system will do it.
Performed during analysis phase	→ Performed during design phase
Identifies requirements, objects, classes and relationships	→ Defines class structures, methods, interfaces and object interactions
Problem oriented	→ Solution oriented
Produces analysis models such as use case diagrams and domain models.	→ Produces design models such as class diagrams, sequence diagrams, design patterns
Does not consider implementation details.	→ Includes implementation details needed for coding.
Main goal is to understand the problem.	→ Main goal is to prepare a blueprint for implementation

Eg: Library Management System.

ODA:

Requirements:

- borrow books
- Search books
- Issue book
- return books
- Register member

Identifying objects:

- Book
- Member
- Librarian
- Loan

OOD:

class: Book

Book
- bookID
- title
- author
+ issueBook()
+ searchBook()

relationships:

- Member borrows book
- Librarian issues book

7) Why is it important to list the requirements before making the actual design? Explain with the help of LMS.

= Listing the requirements before designing the software is essential because it ensures that the system is built according to the user's needs. Requirements define what the system should do, while the design defines how the system will do it.

Importance of listing requirements:

a) Understanding user needs.

→ Helps identify the expectations of users and stakeholders.
Ensures that all required features are included

b) Reduces errors and rework.

→ Detecting missing or incorrect requirements early is less expensive than making changes during development.

Prevents unnecessary redesign and reworking.

c) Provides a clear design foundation:

→ Designers use the requirements as a blueprint to create the system architecture and class design. It ensures the design matches the intended functionality.

d) Improves Communication:

→ Requirements serve as a common reference for customers, analysts, designers and developers. Reduces misunderstandings among team members.

e) Helps in planning and estimation:

→ Enables accurate estimation of project cost, time, resources. Helps identify risks before development begins.

f) Ensures software quality:

→ A well defined requirement list leads to software that satisfies user needs and is easier to test and maintain.

⇒ Example: LMS

Suppose a library wants to develop a library management system. The requirements may include:

- Register library members.
- Add, update and remove books
- Search books by title, author or ISBN.
- Borrow and return books
- Maintain borrowing history

Based on those requirements, the designer identifies the following classes:

- Book • Member • Librarian • Loan

The designer then defines

- attributes (eg. bookID, author, title)
- methods (eg. borrowBook(), returnBook())
- Relationship between classes.

- 8) Explain overall process of Object oriented software development.
- = OOSD is a software development ^{approach} ~~process~~ that models a system as a collection of interacting objects. It applies object oriented principles such as encapsulation, inheritance, polymorphism and abstraction to develop software that is modular, reusable, maintainable and scalable.

The overall process:

i) Object oriented analysis (OOA):

- It is the process of understanding the problem domain and identifying the problem domain and identifying the system requirements by analyzing real world objects, their attributes, behaviors and relationships.

It focuses on what the system should do without considering implementation details.

Activities:

- Gather functional and non functional requirements.
- Identify actors, use cases, objects, classes.
- Determine relationship between objects.
- Prepare analysis models.

~~Contribution~~ Output:

- Software Requirement Specification (SRS)
- Use case diagram, domain model, activity diagram etc.

ii) Object Oriented Design (OOD):

- It transforms the analysis model into a detailed design that serves as a blueprint for coding. It focuses on how the system will work.

Activities:

- Design classes, attributes and methods.
- Define object interactions.
- Design database and user interface.
- Apply design patterns where necessary.

Output :

→ class diagram, sequence diagram, communication diagram

iii) Object oriented modeling (OOM)

→ It is the process of representing the software system using UML diagrams. It provides a visual representation of both the structure and behavior of the system.

Improves communication among developers and stakeholders.

Activities:

→ Create structural models.

→ Create behavioral models

→ Document objects relationships and interactions.

Common UML diagrams:

→ Use case, class, object, sequence, communication, activity, state.

iv) Object oriented implementation:

→ It is the phase where the designed classes and objects are translated into program code using an object oriented programming language such as Java, C++ etc.

It converts the design into a functional software system. Ensures the software meets the specified requirements

Activities:

→ Write source code.

→ Integrate modules.

→ Perform unit, integration and System testing.

→ Debug and deploy the software.

Output:

→ Working software application.